

Relazione Progetto MapReduce - libmr

Framework per analisi di file di testo

Maggio 2026

1 Introduzione

Il progetto consiste nella realizzazione di un framework denominato `libmr`, scritto in linguaggio C, che implementa il paradigma MapReduce per l'analisi di file testuali su un singolo calcolatore. Il framework gestisce in modo autonomo la creazione di processi, la comunicazione tramite pipe e l'orchestrazione di thread C11, permettendo all'utente di concentrarsi esclusivamente sulla logica di *mapping* e *reducing*.

Il framework è ora completamente operativo e implementa tutte le funzionalità richieste, inclusa la pipeline multiprocesso e l'esecuzione multithreaded per la parallelizzazione del carico di lavoro.

2 Architettura del Sistema

L'architettura di `libmr` si basa su una pipeline di tre processi distinti che comunicano in modo unidirezionale tramite pipe POSIX:

- **Processo Principale:** Agisce come coordinatore. Legge i file di input (singoli file o intere directory), invia le righe al processo Mapper tramite la pipe `main_to_mapper` e riceve i risultati finali dal processo Reducer tramite la pipe `reducer_to_main`.
- **Processo Mapper:** Riceve righe di testo dal Main. Un thread controller legge dalla pipe e inserisce le righe in una coda produttore-consumatore. Un pool di thread *worker* (configurabile tramite `mr_attr_t`) preleva le righe, applica la funzione di *mapping* fornita dall'utente e invia le coppie `<token, valore>` risultanti al processo Reducer tramite la pipe `mapper_to_reducer`.
- **Processo Reducer:** Riceve le coppie dal Mapper. Un thread controller raggruppa i valori per chiave utilizzando una tabella hash. Una volta terminato l'input, i gruppi di valori vengono inseriti in una coda per i thread *worker* del reducer. Questi applicano la funzione di *reducing* e inviano i risultati finali al processo Principale.

3 Comunicazione e Sincronizzazione

La comunicazione tra processi avviene tramite un protocollo binario semplificato implementato in `src/protocol.c`, che garantisce l'invio corretto di stringhe e metadati attraverso le pipe.

All'interno dei processi Mapper e Reducer, la sincronizzazione è gestita tramite:

- **Code Bloccanti:** Implementate in `src/queue.c` utilizzando `mtx_t` e `cnd_t` (C11 threads) per gestire la distribuzione del lavoro tra thread controller e thread worker.
- **Mutex di Output:** L'accesso alle pipe di scrittura condivise tra più thread worker è protetto da mutex per evitare l'interleave dei messaggi.

4 Sistema di Logging

Il sistema di logging è sincronizzato e persistente. Le sue caratteristiche principali includono:

- **Sincronizzazione:** Utilizzo di semafori POSIX (`sem_t`) per garantire che le scritture sul file di log siano atomiche, anche quando provengono da processi diversi.
- **Formato:** Ogni riga di log segue il formato:

```
<timestamp> <pname>[<pid>] <tname> <level>: <message>
```

- **Configurabilità:** Il livello di log e il file di destinazione sono personalizzabili tramite gli attributi del framework.

5 Test e Validazione

La robustezza del framework è stata verificata attraverso una suite di test completa:

- **Unit Tests:** Utilizzando **Unity**, sono stati testati i singoli moduli (code, protocollo, gestione attributi, logging).
- **Integration Tests:** Sono stati realizzati test di integrazione che simulano l'intero ciclo di vita di un job MapReduce, verificando la corretta terminazione dei processi e l'assenza di memory leak o deadlock.
- **Esempi Reali:** L'esempio `word_count` incluso dimostra l'efficacia del framework su file di testo reali, producendo risultati coerenti con quelli ottenuti tramite strumenti standard (es. `wc`, `sort`).

6 Conclusioni

`libmr` rappresenta un'implementazione efficiente e modulare del paradigma MapReduce per sistemi POSIX. L'uso di processi separati garantisce isolamento, mentre l'impiego di thread all'interno di ciascun processo permette di sfruttare appieno le architetture multicore. Il framework si è dimostrato stabile e performante, rispettando tutti i requisiti di specifica iniziali.